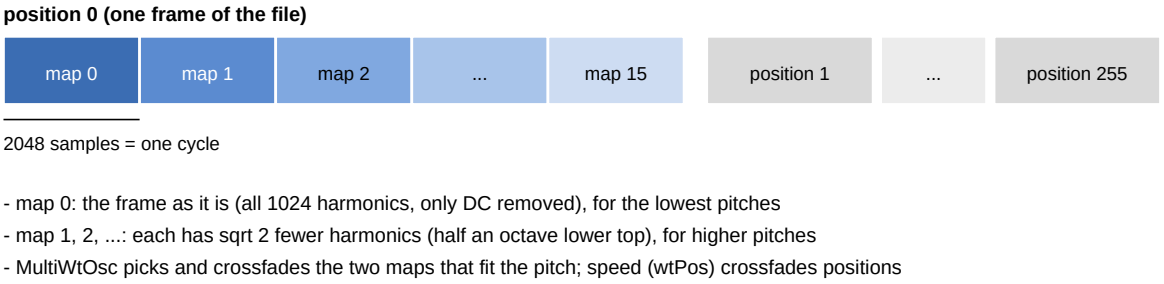
Wavetables without aliasing

How to use `NebPatch.loadWavetables` and `MultiWtOsc` in a SuperCollider patch on the Nebulae

1. What it does

A wavetable file holds many single cycles (frames) of a waveform, e.g. 256 frames of 2048 samples as made by Serum. Played as they are, high notes alias: harmonics above half the sample rate (Nyquist, 24 kHz) fold back down as inharmonic noise. The top notes of `wave_tables.scd` had aliasing at -5 dB, almost as loud as the sound itself.

`loadWavetables` prepares every frame once into 16 band-limited copies (*maps*). `MultiWtOsc` plays the copy that fits the current pitch, so a harmonic is only removed when it would alias. The result is saved in a cache and reused until the audio file changes.



Layout of one prepared buffer. Based on WavetablePrep and MultiWtOsc from ddwWavetableSynth by James Harkins (GPL v3), with Nebulae fixes (section 7).

2. Preparing the audio files

Rule	Why
`mono`	Stereo files are refused with an error in /tmp/sclang.log.
`frames of 2048 samples`	Every 2048 samples is one cycle = one position. Serum, Vital and most wavetable editors export like this. A file of 256 x 2048 samples has 256 positions; any whole number works (wt_3 has 68).
`any sample rate / bit depth`	Only the samples are used: 16 bit, 24 bit and float all work.
`name it wt_*.wav`	wave_tables.scd loads the files matching ^wt_.*wav$. Your own patch can use any regular expression.
`put it in audio/ on the USB stick`	It ends up in /home/alarm/audio/ (NebInterface.audioPath).

3. The smallest patch

This is `Supercollider/Wave Tables/wave_tables.scd`, shortened. Everything happens inside `s.waitForBoot`, because `loadWavetables` waits there until the buffers are ready.

```
s.waitForBoot {
    var neb = NebInterface.init(s);                        // gives the NebPatch
    var wtSize = 2048, numMaps = 16, ratio = 2.sqrt;       // = loadWavetables' defaults
    var bufs = neb.loadWavetables("^wt_.*wav$", wtSize, numMaps, ratio, true);
    if (bufs.isEmpty) { "no wt_*.wav".warn } {
        {
            var freq = LinExp.kr(NebPitch.kr(0, 1), 0, 1, 30, 2000);
            var sigs = bufs.collect { |b|
                var last = NebPatch.wavetablePositions(b, wtSize, numMaps) - 1.001;
                MultiWtOsc.ar(freq, NebSpeed.kr(0, last),     // speed = position
                    bufnum: b, numTables: numMaps, wtSize: wtSize, ratio: ratio,
                    interpolation: 4,                         // cubic
                    mapOffset: 1)                             // needed, see section 4
            };
            SelectX.ar(NebWindow.kr(0, bufs.size - 1), sigs) ! 2 * 0.1
        }.play;
    };
    NebInterface.ready(s);
};
```

Buffers can only be loaded through the object that `NebInterface.init` returns, so they always come after `init`. If `init` runs again, the old object throws an error instead of silently working on freed buffers.

4. Arguments

neb.loadWavetables(regex, wtSize, numMaps, ratio, storeBuff, path, fudgeFactor) returns an Array of Buffers, one per matching file, in file name order.

Argument	Default	Meaning
`regex`	-	Which files in the folder, e.g. "^wt_.*wav$".
`wtSize`	2048	Samples per frame (one cycle).
`numMaps`	16	Number of band-limited copies per frame.
`ratio`	sqrt 2	Step between two copies: each has ratio fewer harmonics. sqrt 2 = half an octave.
`storeBuff`	false	true: the buffers are kept for the next patch (NebInterface.storedBuffers).
`path`	audioPath	Folder to read from, default /home/alarm/audio/.
`fudgeFactor`	0	0: map 0 is the unfiltered frame. Leave it at 0.

MultiWtOsc.ar must get the same layout, or it reads the wrong copies:

MultiWtOsc argument	Value	Note
`bufnum`	one buffer from loadWavetables	
`wtSize`	same as loadWavetables	2048
`numTables`	= numMaps	16. (MultiWtOsc calls the maps 'tables'.)
`ratio`	same as loadWavetables	2.sqrt
`mapOffset`	1	Uses each copy only up to the pitch where it would alias. With 0 (upstream) the crossfades alias.
`interpolation`	4 (cubic) or 2 (linear)	Cubic: cleaner at low pitches, about twice the CPU.
`wtPos`	0 .. positions - 1.001	NebPatch.wavetablePositions(buf, wtSize, numMaps) gives the number of positions of each file. Higher values read beyond the file.

5. The cache

`Where`	/home/alarm/cache/wavetables/ (NebInterface.wavetableCachePath). Not in audio/ (emptied on every USB load) and not in QB_Nebulae_V2/ (replaced by firmware updates).
`File name`	<name>_<md5>_<wtSize>x<numMaps>r<ratio>f<fudge>.wav, e.g. wt_1_3f2a..._2048x16r1.414f0.0.wav
`Reused when`	the audio file has the same content (md5) and the same settings. Copying the same file from USB again does not prepare it again.
`Prepared again when`	the file content or a setting changed (the old cache file of that file is deleted), or the cache file is incomplete (e.g. power off while writing).
`Time`	About 0.6 s per 256 positions on a PC, a few seconds on the RPi, only the first time. Loading from the cache takes a fraction of a second. Nebulae waits about 25 s for a patch.
`Size`	positions x numMaps x 2048 x 4 bytes: 34 MB for a 256-position file, both on the SD card and in scsynth's memory. The RPi has 3.4 GB free.
`Start again`	Delete the files in the cache folder; they are made on the next load.

Same file as a normal sample: the audio file itself is never changed. `neb.loadAudio("^wt_1.wav$")` gives the original, full file, e.g. to play it at its own pitch. The two are stored under different names and don't mix.

6. Measured quality

At 48 kHz with wt_1.wav, position 100. Aliasing = energy that is not a harmonic of the note, relative to everything. Brightest = highest harmonic still there (Nyquist is 24 kHz).

Setup	Aliasing 65 Hz - 2 kHz	Brightest
`Before: raw file, numTables 1`	-57 .. -5 dB (2 kHz: -5 dB)	everything, but aliased
`Upstream WavetablePrep (ratio 2, 8 maps, fudge 0.5)`	-82 .. -41 dB	up to 23 kHz
`Ratio 2, 8 maps, fudge 1`	-88 .. -67 dB	10 - 17 kHz (up to an octave duller)
`Now: ratio sqrt 2, 16 maps, mapOffset 1, cubic`	-84 .. -64 dB	17 - 23 kHz

Below about 33 Hz the unfiltered map 0 keeps the whole frame (at 16 Hz: 16 dB more at 11.8 - 16.6 kHz than with a filtered map 0). There, interpolation leaves mirror images at 17 - 24 kHz at about -34 dB; no 4-point interpolation can remove them.

7. What was changed in the extensions

File	Change
`MultiWtOsc/WavetablePrep.sc`	New: the non-GUI part of WavetablePrep (no Image, Color or GUI, so it runs headless). Added readSync (runs in the patch's Routine). A map that covers the whole table is left unfiltered.
`MultiWtOsc/MultiWtOsc.sc`	Interpolation wraps inside each table. Upstream interpolated the last sample of a cycle with the first sample of the next table: a small jump every cycle, aliasing at -25 dB even with prepared tables (now -84 dB). Index and fraction are kept apart, so large buffers lose no precision. Cubic (4) wraps too. New argument mapOffset.
`NebInterface/NebInterface.sc`	NebPatch.loadWavetables, NebPatch.wavetablePositions, NebInterface.wavetableCachePath.

8. Problems

What you see	Cause and fix
`Warning: no wt_*.wav in /home/alarm/audio/`	No matching files. Check the names and the audio/ folder on the stick. The patch still starts, but plays nothing.
`Error: ... has 2 channels, a wavetable file must be mono`	Export the file as mono.
`Noise or wrong sound at high notes`	MultiWtOsc doesn't match: check numTables = numMaps, ratio, and mapOffset: 1.
`Clicks or garbage when turning speed to the end`	wtPos goes beyond the file: use NebPatch.wavetablePositions(...) - 1.001 as the maximum.
`CPU too high`	Use interpolation: 2 (linear). It halves the reads; the maps still prevent aliasing.
`Error: NebPatch: NebInterface.init was called again`	Use the object from the last NebInterface.init; buffers from before it are freed.